

DEEP COMPRESSION MLP

Table of Contents

Table of Contents.....	1
1. Introduction	2
2. Background & Motivation	2
3. Model Architecture	2
4. Compression Pipeline	3
4.1 Phase 1 — Baseline Training	3
4.2 Phase 2 — Magnitude Pruning	3
4.3 Fine-Tuning After Pruning.....	4
4.4 Phase 3 — K-Means Quantization.....	4
4.5 Phase 4 — Huffman Encoding.....	5
5. Results Summary.....	6
6. Key Observations.....	6
7. Project Structure	7
8. Dependencies & Reproduction.....	Error! Bookmark not defined.
9. Reference	Error! Bookmark not defined.

1. Introduction

Deep Compression is a neural network compression framework introduced by Song Han, Huizi Mao, and William J. Dally, which received the Best Paper Award at ICLR 2016. The technique achieves dramatic reductions in model storage size by chaining three complementary compression stages: magnitude-based pruning, k-means weight quantization, and Huffman encoding.

This report documents the application of the Deep Compression pipeline to a Multi-Layer Perceptron (MLP) trained on the MNIST handwritten digit recognition dataset. The experiment demonstrates that the model can be compressed by more than 31 times — from 1.78 MB to 0.057 MB — while preserving 97.35% classification accuracy, essentially identical to the 97.33% baseline.

Key Objective

Apply the full Deep Compression pipeline (Pruning → Quantization → Huffman Encoding) to an MLP on MNIST and quantify the compression vs. accuracy trade-off at each stage.

2. Background & Motivation

Modern deep learning models contain tens to hundreds of millions of parameters stored as 32-bit floating point numbers. This creates significant challenges for deployment on resource-constrained devices such as mobile phones and embedded systems, which have limited memory, bandwidth, and battery capacity.

Deep Compression addresses this by exploiting two key properties of trained neural networks:

- **Weight redundancy:** Many weights are near zero and can be removed without harming accuracy.
- **Weight clustering:** Active weights tend to cluster around a small number of values, allowing many weights to share a single representative.
- **Non-uniform weight distribution:** Pruned weights (zero) dominate the weight distribution, enabling highly efficient entropy coding.

The pipeline processes these observations in sequence, with each stage building on the output of the previous one, achieving compounding savings that far exceed what any single technique could accomplish alone.

3. Model Architecture

The baseline model is a fully-connected MLP designed for MNIST classification. Each linear layer is implemented as a `modified_linear` module (in `compression/linear.py`), which extends `torch.nn.Linear` to natively support pruning masks and quantized weight look-up tables.

```
mnist (top-level wrapper)
├─ Flatten
├─ Sequential (classifier)
│   ├── modified_linear [784 → hidden] ← compression-aware
│   ├── ReLU
│   ├── modified_linear [hidden → hidden]
│   ├── ReLU
│   ├── modified_linear [hidden → hidden]
│   ├── ReLU
│   └─ modified_linear [hidden → 10]
└─

Total Parameters: 466,442
Trainable Weights: 465,408 (classifier layers only)
```

The `modified_linear` layer stores a binary mask of the same shape as the weight matrix. During the forward pass, weights are multiplied element-wise by the mask, ensuring pruned connections contribute zero activation. During quantization, the weight matrix is replaced by an index tensor pointing into a small per-layer codebook.

4. Compression Pipeline

4.1 Phase 1 — Baseline Training

The model is trained from scratch on the MNIST training set for 5 epochs using cross-entropy loss and a standard optimizer. This establishes the performance ceiling that the compression pipeline must preserve.

Epoch	Loss	Accuracy	Observation
1	0.4633	94.41%	Initial convergence
2	0.1458	95.16%	Rapid improvement
3	0.0974	96.75%	Stabilizing
4	0.0713	96.58%	Minor fluctuation
5	0.0569	97.33%	Best baseline

The baseline achieves 97.33% test accuracy at convergence, which serves as the reference point for all subsequent compression stages.

4.2 Phase 2 — Magnitude Pruning

Magnitude pruning identifies and removes weights whose absolute value falls below a global threshold. The intuition is that small weights contribute negligibly to the model's predictions and can be zeroed out without meaningful accuracy degradation.

Pruning Mechanics

A binary mask is applied to each weight tensor. Weights below the magnitude threshold are set to zero in the mask. The model is re-evaluated and subsequently fine-tuned with the mask frozen, preventing pruned connections from recovering.

Sparsity Report — Before Pruning:

- Total Weights: 465,408
- Active Weights: 232,704 (pre-existing mask from initialisation)
- Zeroed Weights: 232,704
- Sparsity: 50.00%

Sparsity Report — After Magnitude Pruning:

- Total Weights: 465,408
- Active Weights: 224,941
- Zeroed Weights: 240,467
- Global Sparsity Achieved: 51.67%

4.3 Fine-Tuning After Pruning

After pruning, the sparse model is fine-tuned for 3 additional epochs to allow remaining active weights to compensate for removed connections. The pruning mask is held constant during fine-tuning so that zeroed weights cannot be restored.

Epoch	Loss	Accuracy	Delta vs. Baseline
1	0.0321	97.63%	+0.30%
2	0.0240	97.72%	+0.39%
3	0.0189	97.71%	+0.38%

4.4 Phase 3 — K-Means Quantization

Weight quantization reduces the precision of the active weight values by replacing them with cluster centroid representatives. Using k-means clustering with $k=256$ (8-bit), each layer's active weights are grouped into at most 256 unique values. Each weight is then replaced by its cluster index, and only the codebook of 256 centroids needs to be stored per layer in full precision.

Layer	Unique Weights	Target (8-bit)	Status
classifier.0	232	256	Active — fits within 8-bit
classifier.2	211	256	Active — fits within 8-bit
classifier.4	195	256	Active — fits within 8-bit
classifier.6	195	256	Active — fits within 8-bit

All four layers had fewer than 256 unique active weights to begin with, meaning 8-bit quantization introduces no additional precision loss in this case — the quantized values are lossless representations.

Storage Savings from Quantization

Baseline (32-bit float): 1.7793 MB

Quantized (8-bit index): 0.4448 MB

Compression Ratio: 4.0x smaller

Post-quantization test accuracy: 97.70% — a negligible drop of 0.01% from the fine-tuned model.

4.5 Phase 4 — Huffman Encoding

Huffman encoding is an entropy coding scheme that assigns variable-length binary codes to symbols based on their frequency. Frequent symbols receive short codes; rare symbols receive longer codes. Applied to the quantized weight indices, Huffman encoding exploits the extremely skewed distribution of weight values — the zero-weight index appears 465,408 times and receives a single-bit code.

Weight Value	Frequency	Code Length (bits)	Note
0.0000	465,408	1	Most frequent (zero weights)
-0.0165	1	11	Rare weight
-0.0228	1	11	Rare weight
-0.0152	1	11	Rare weight
0.0384	1	11	Rare weight

The Huffman dictionary contained 1,035 unique weight clusters across all layers. The extreme dominance of the zero-weight value (encoded as '1' in just 1 bit) enables the final stage to achieve most of its savings.

Final Compression Result

Huffman Encoded Size: 0.0568 MB

Original Baseline Size: 1.7793 MB

FINAL COMPRESSION RATIO: 31.3x smaller

5. Results Summary

The table below consolidates all metrics across the complete pipeline:

Stage	Accuracy	Model Size	Sparsity	Compression
Baseline Training (32-bit)	97.33%	1.7793 MB	—	1×
After Pruning	97.33%	—	50.00%	~2×
After Fine-tuning	97.71%	—	51.67%	~2×
After Quantization (8-bit)	97.70%	0.4448 MB	51.67%	4.0×
After Huffman Encoding	97.35%	0.0568 MB	51.67%	31.3×

The compression ratio compounds across all stages: pruning eliminates redundant connections, quantization reduces the bit-width of remaining weights, and Huffman encoding further compresses the resulting sparse integer representation. The model accuracy is preserved throughout the pipeline, with only 0.02% total accuracy loss from baseline to final compressed model.

6. Key Observations

- Pruning alone achieves ~2x compression with no accuracy loss when combined with fine-tuning.
- Quantization delivers a clean 4x compression over the baseline at negligible accuracy cost, owing to the natural clustering of trained weights.
- The largest compression gain comes from Huffman encoding, which exploits the highly imbalanced frequency distribution of quantized indices — particularly the dominance of the zero-weight cluster.
- The three stages are synergistic: pruning creates the sparse distribution that makes Huffman coding extremely effective; quantization reduces the codebook that Huffman operates over.

- The final model retains 97.35% accuracy against a 97.33% baseline — effectively no degradation.
- The entire pipeline runs on CPU, making it accessible without GPU infrastructure.

7. Project Structure

```
deep-compression-mlp/  
├── main.py           # Pipeline entry point  
├── models/  
│   └── mnist.py     # MNIST MLP model definition  
└── compression/  
    └── linear.py     # modified_linear: pruning & quantization-aware  
layer
```